

Lecture 4: Empirical Risk and Gradient Descent

Announcements

- HW1 is due Wednesday 9/16

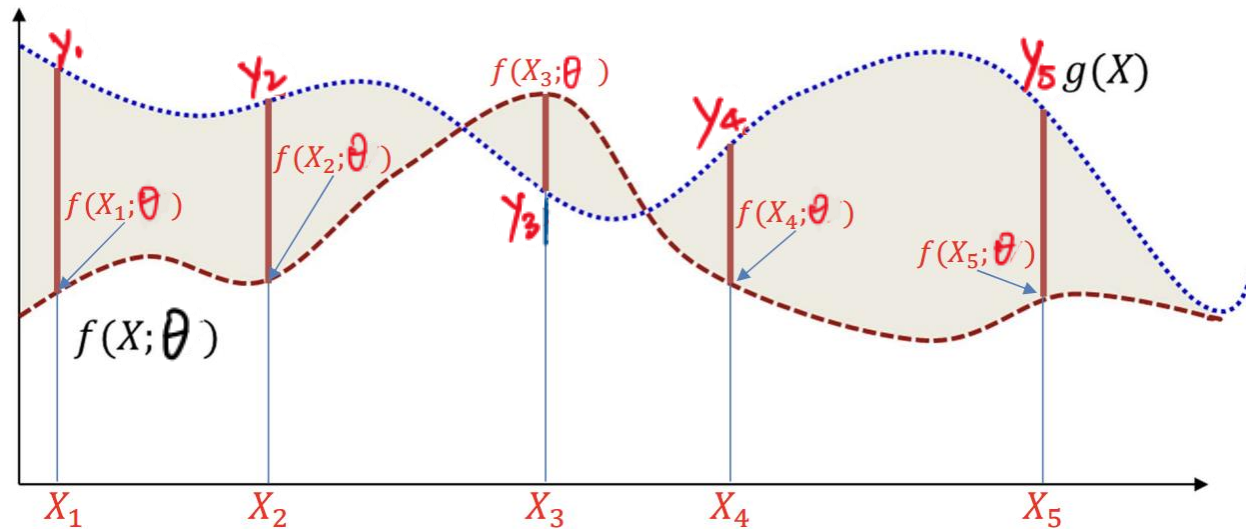
Recap: Data Generating Process

- Enable us to describe how ML algorithms are learning
 - In the language of
 - Generalization error, **True risk**, **Empirical risk**...
- Model randomness/uncertainty in the dataset
- Can think of data as generated according to some unknown function f :

$$Y = f(X) + \epsilon$$

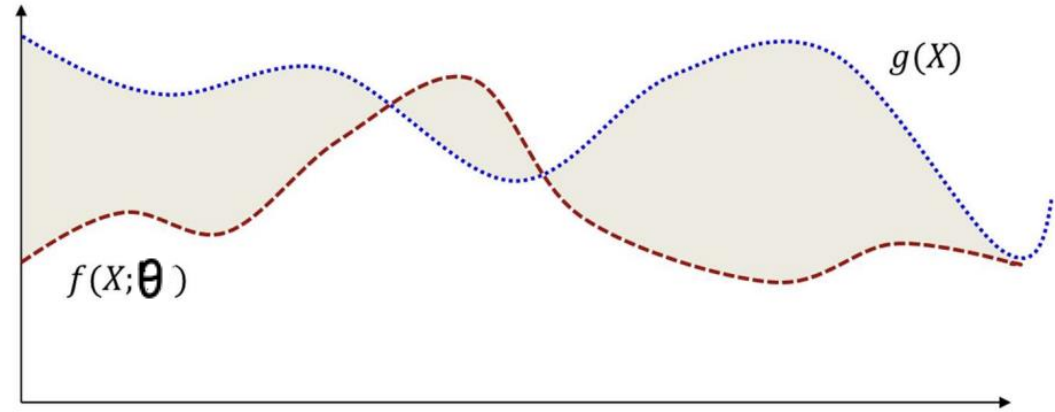
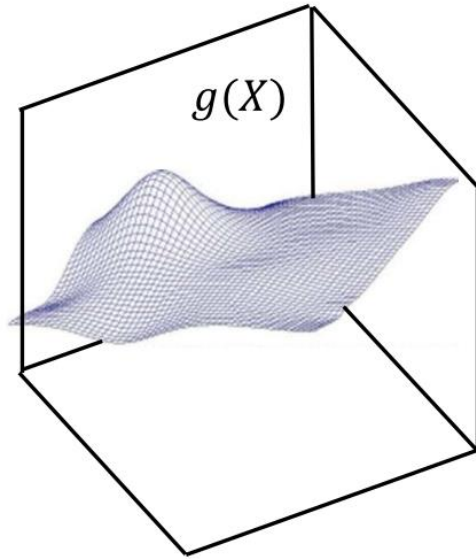
- Use probability and statistics to analyze ML algorithms

How to learn an ML model?



- The **blue curve** is the true data-generating mechanism that we are trying to learn
- The **red curve** is the current function that I fitted on the data

Problem: $g(X)$ is unknown



- Function must be fully specified in order to compute

$$\int_{-\infty}^{\infty} \text{error}(f(X; \theta), g(X)) dX$$

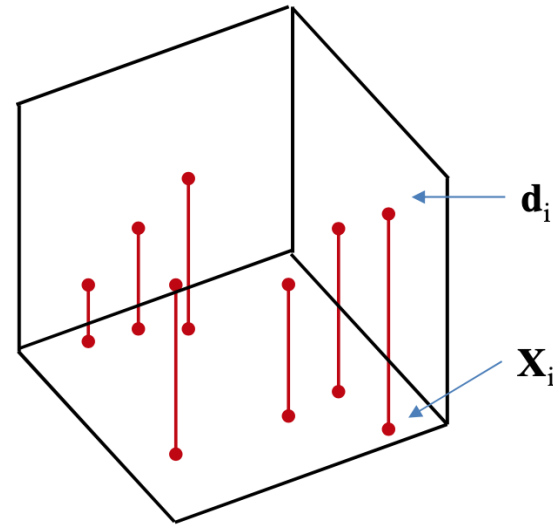
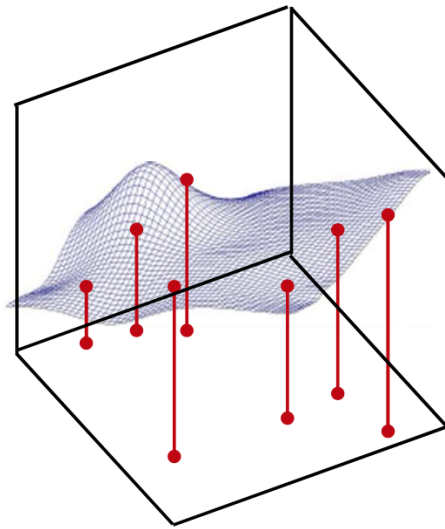
- Aka True risk:

$$R(\theta) = E_{(X,Y) \sim P} \text{error}(f(X; \theta), g(\theta))$$

DGP

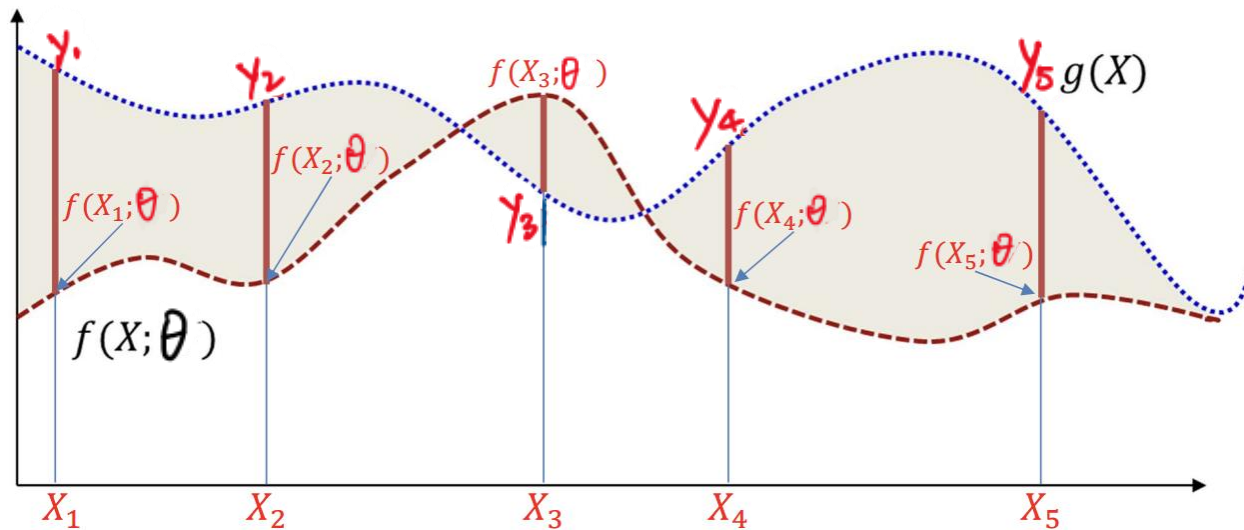
- In practice we will not have such specification

Solution: Sampling the function



- Sample $g(\mathbf{x})$
 - Basically, get input output pairs for a number of samples of input
 - Very easy to do in most problems: just gather training data
 - We must learn the entire function from these few examples
 - The “training” samples

The Empirical Error



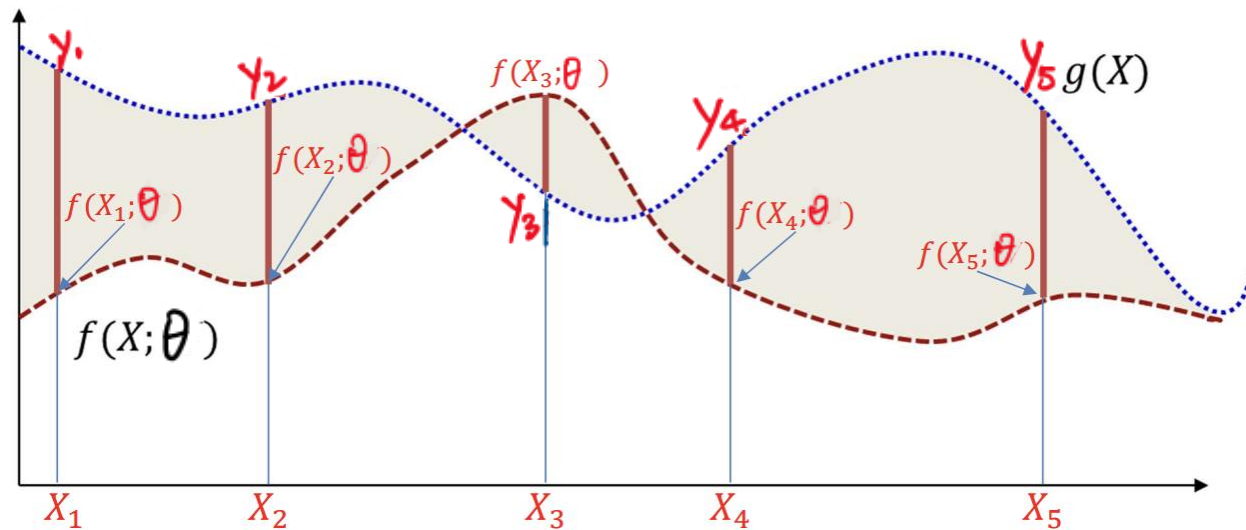
- The empirical estimate of the error is the average error over the training samples

$$\text{EmpiricalError}(\theta) = \frac{1}{N} \sum_{i=1}^N \text{error}(f(X_i; \theta), y_i)$$

- Estimate model parameters to minimize this average error instead

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \text{EmpiricalError}(\theta)$$

The Empirical Error



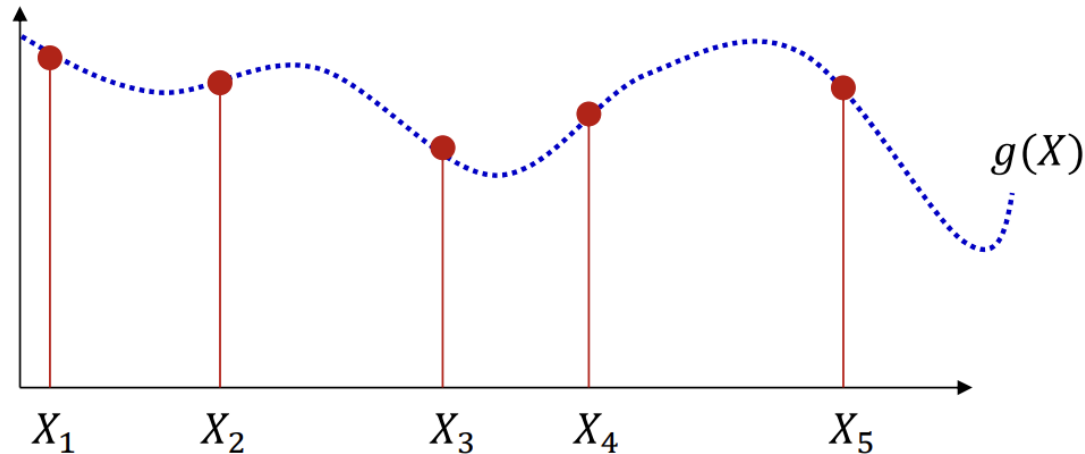
- The **empirical estimate** of the error is the average error over the training samples

$$LOSS(\theta) = \frac{1}{N} \sum_{i=1}^N error(f(X_i; \theta), y_i)$$

- Estimate model parameters to minimize this average error instead

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \quad LOSS(\theta)$$

Learning the function from training samples



- Estimate the model parameters to “best fit” the training points
 - Hopefully, the resulting function generalizes to places where we don’t have training samples
 - More about generalization errors/training and testing split later!

Supervised Learning Problem Restated

- Given a training set of input-label pairs

$$\mathcal{D} = \{(X_i, Y_i) | i = 1, 2, \dots, n\}$$

- Select a model class parametrized by θ
- Quantify the error on the i^{th} sample $error(f(X_i; \theta), Y_i)$
- Calculate the empirical risk

$$Loss(\theta) = \frac{1}{N} \sum_{i=1}^n error(f(X_i; \theta), Y_i)$$

- Estimate the parameters to minimize the empirical risk

$$\hat{\theta} = \arg \min_{\theta} Loss(\theta)$$

- i.e., minimize the *empirical risk* over the drawn samples

Select all correct statements

0

To optimize the loss function, we want $error(f, g)$ to be differentiable with respect to f

0

We call $error(f, g)$ the empirical risk because it is only an empirical approximation to the true risk (the shaded area between f and g)

0

For a given training set, the loss is only a function of θ

0

Minimizing the loss function $loss(\theta)$ with respect to θ is a problem of function minimization, which is an instance of optimization and can be solved using gradient descent

0

The loss function quantifies the mismatch between the model output and the target function

0

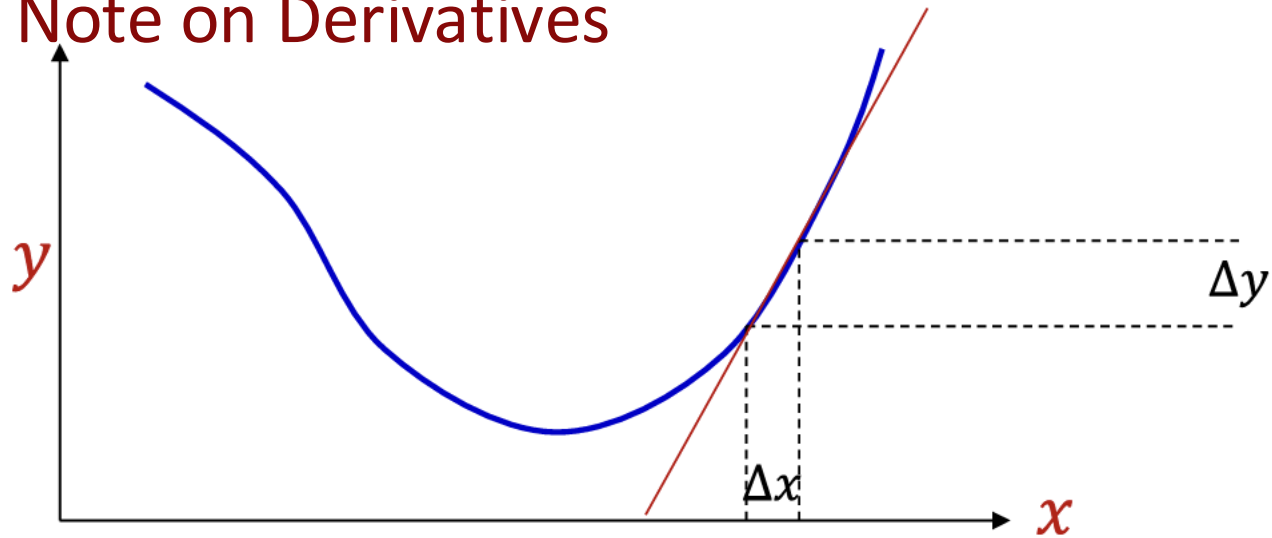
The error function should satisfy the following properties: 1. $error(f(X_i; \theta), g(X_i)) > 0$ if $f(X_i, \theta) \neq g(X_i)$; 2. $error(f(X_i, \theta), g(X_i)) = 0$ if $f(X_i, \theta) = g(X_i)$

0

Recap

- Data generating distribution and empirical risk
- When the dataset is **fixed**, the loss function is only a function of the model parameter θ
 - This is an optimization problem: can be solve by first finding the point where $\nabla_X f(X) = 0$, and then checking the property of the Hessian
- Finding the roots of $\nabla_X f(X) = 0$ is not easy! (The function might have an intractable form)
- Iterative solutions are needed!
 - Gradient descent

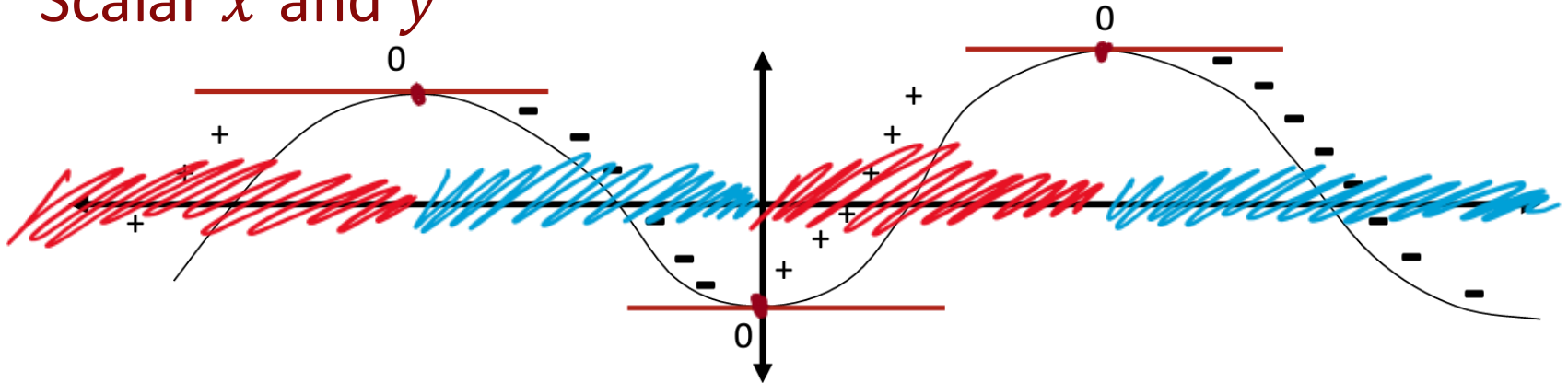
A Brief Note on Derivatives



A derivative of a function, $f(x)$, at any point x tells us how much a small increment to the argument of the function, Δx , will increment the value of the function, Δy :

- $\Delta y = \alpha \Delta x$ where α is the derivative of $f(x)$ at x
- At a fine enough resolution, any smooth, continuous function is locally linear at any point

Scalar x and y

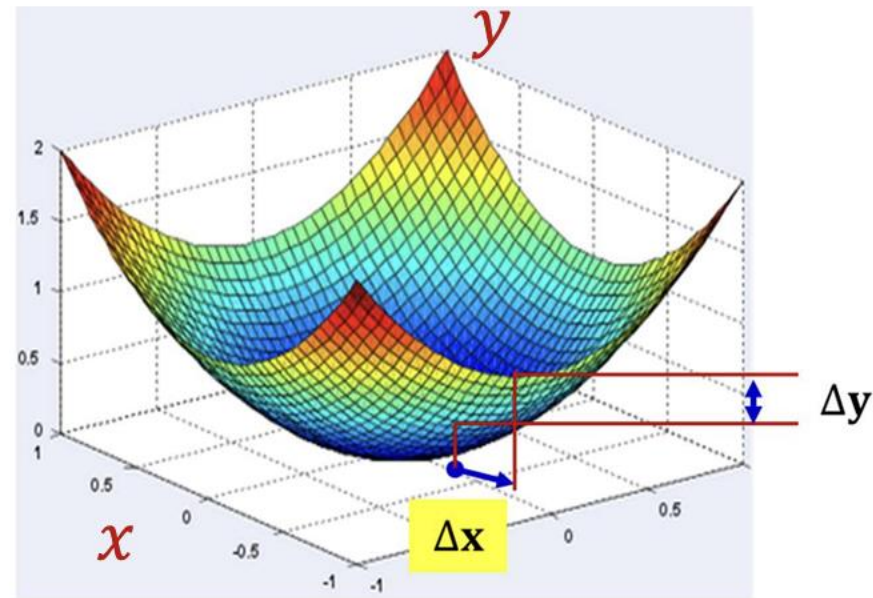


- Derivative $f'(x)$ is the rate of change of the function at x
- Measures how fast $f(x)$ increases with x
- Magnitude of $f'(x) \rightarrow$ steepness of the curve at x
- Positive where a small increase in x results in an **increase** of $f(x)$
- Negative where a small increase in x results in a **decrease** of $f(x)$
- **0** where the function is locally flat (neither increasing nor decreasing)

Multivariate X Scalar y

$$\Delta y = \alpha_1 \Delta x_1 + \alpha_2 \Delta x_2 + \cdots + \alpha_d \Delta x_d$$

- $\alpha_i = \frac{\partial y}{\partial x_i}$: partial derivative
indicates how y increments when
only x_i is incremented



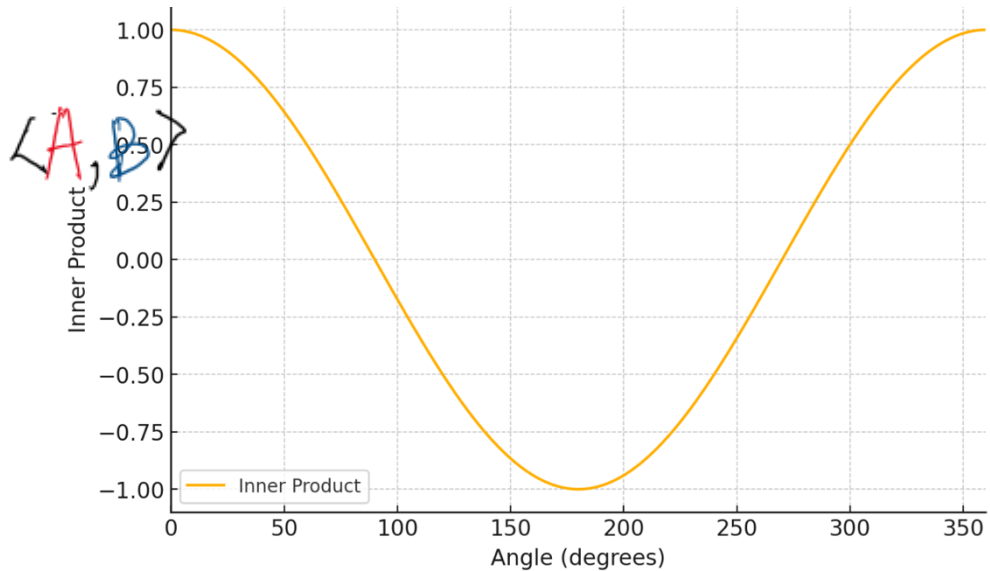
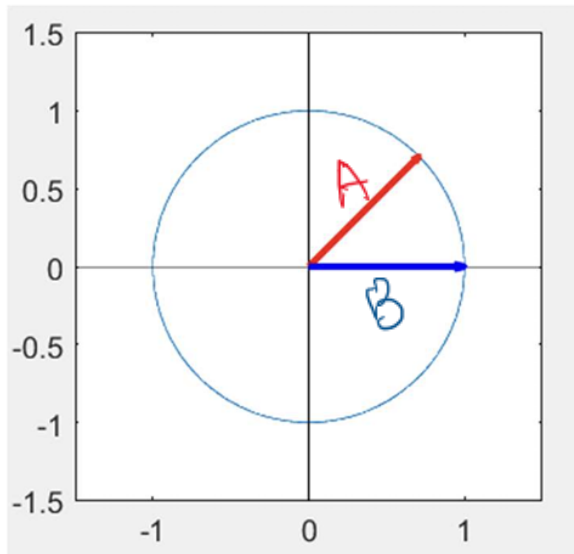
- Gradient $\nabla_X y$: a *column* vector containing all partial derivatives

$$\nabla_X y = \left[\frac{\partial y}{\partial x_1} \quad \cdots \quad \frac{\partial y}{\partial x_d} \right]^T$$

- ΔX : a column vector containing Δx_i 's
- Write $\Delta y = \langle \nabla_X y, \Delta X \rangle = \nabla_X y^T \Delta X$

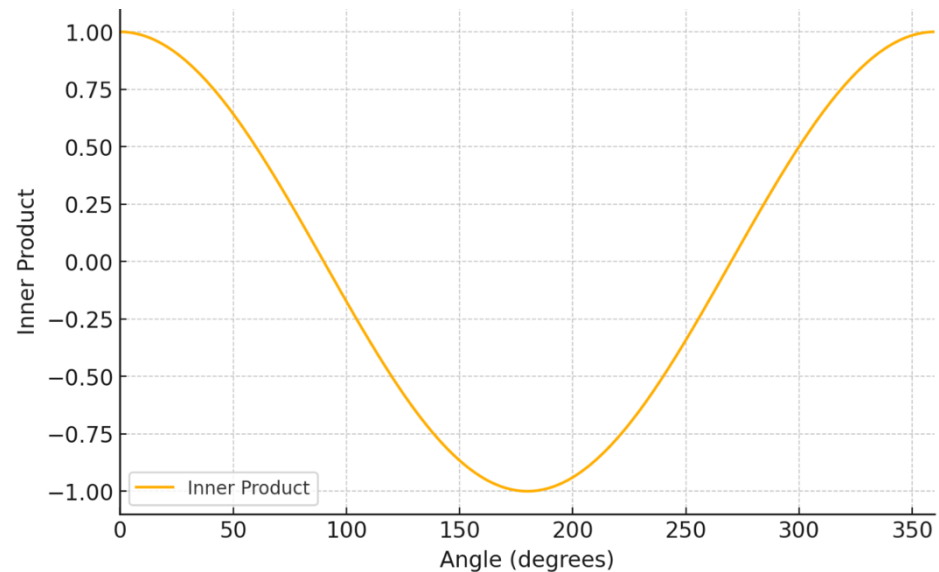
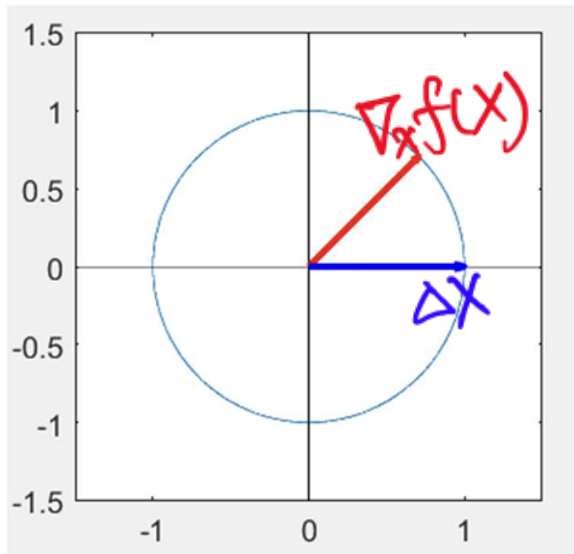
To understand its behavior, let's consider a well-known
property of inner products

A Well-Known Vector Property



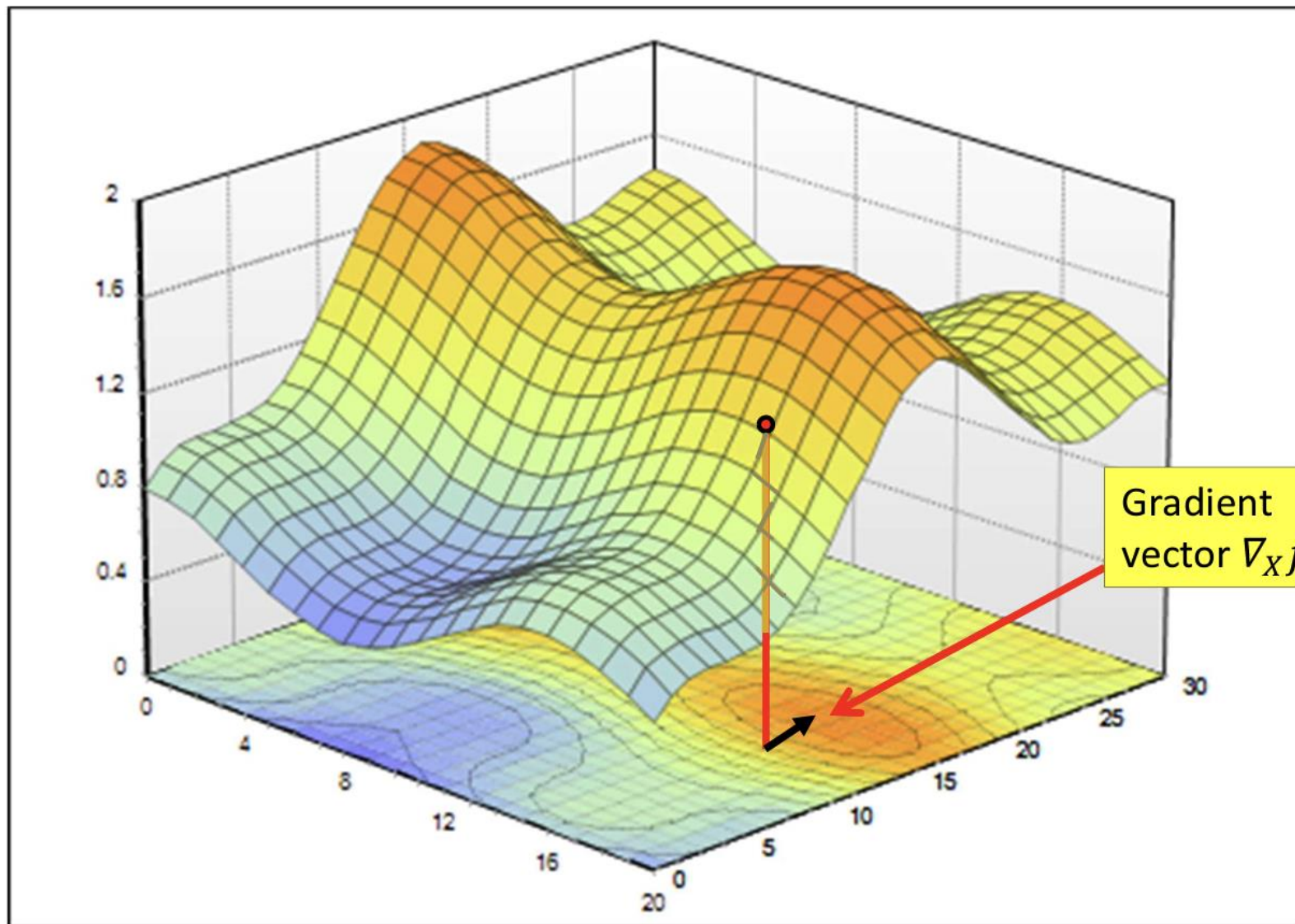
- The inner product between two vectors of fixed lengths is maximum when the two vectors are aligned
- i.e., when angle equals 0

A Well-Known Vector Property

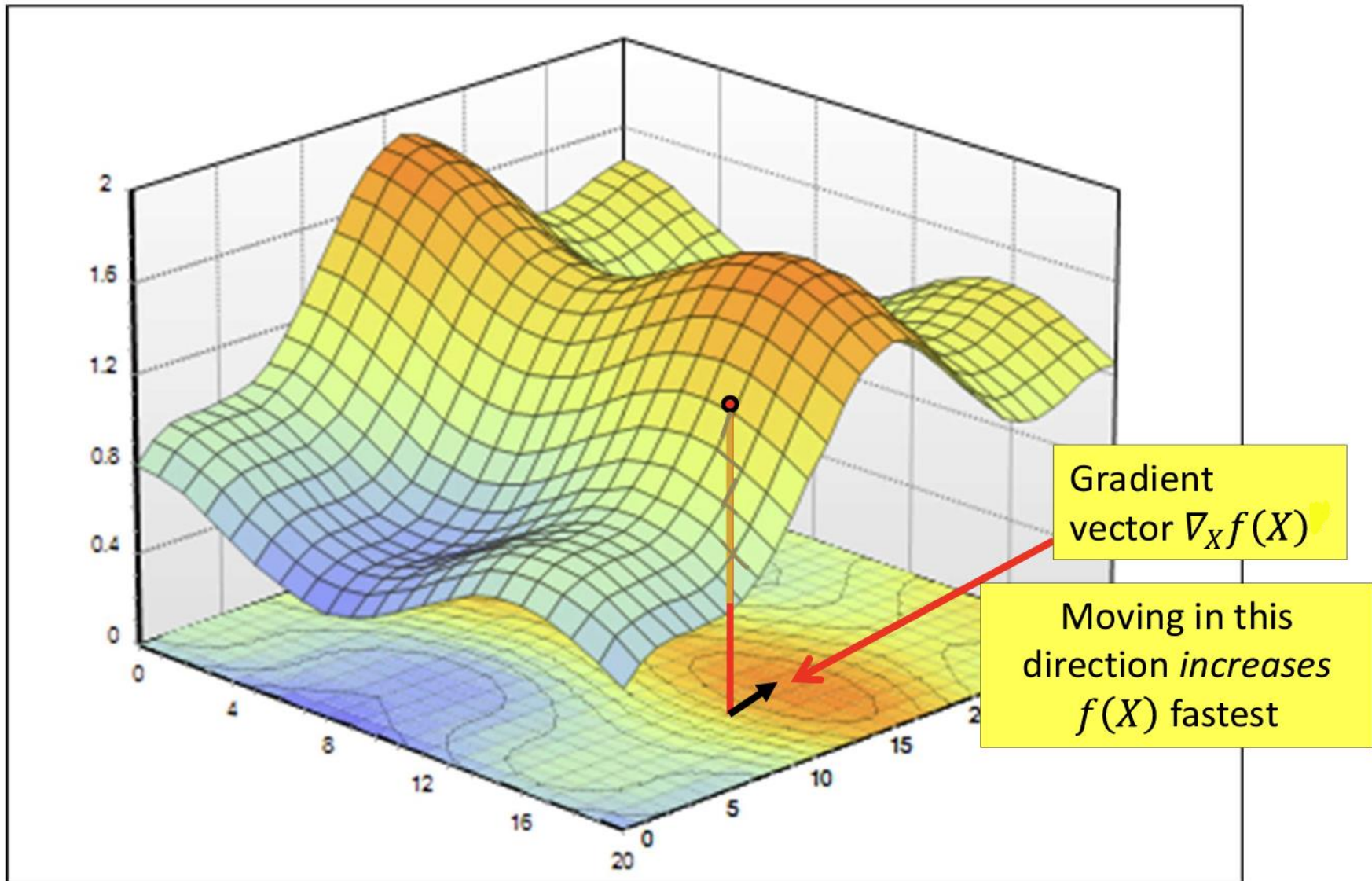


- $\Delta y = \langle \nabla_X y, \Delta X \rangle$
- For an increment ΔX of any given length Δy is max if ΔX is aligned with $\nabla_X y$
- The function $f(X)$ increases most rapidly if the input increment ΔX is exactly in the direction of $\nabla_X y$
- The gradient is the direction of fastest increase in $f(X)$

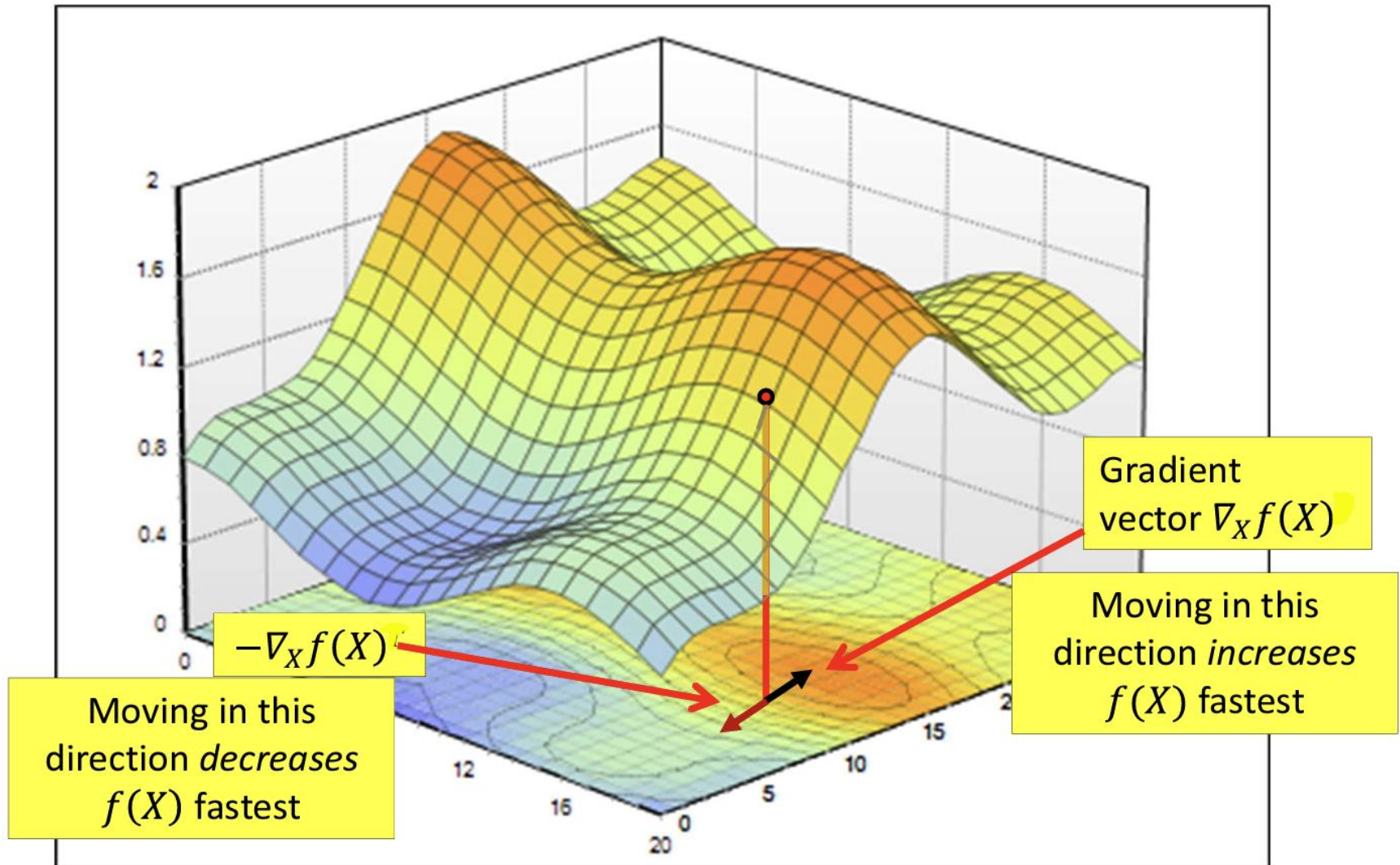
Gradient



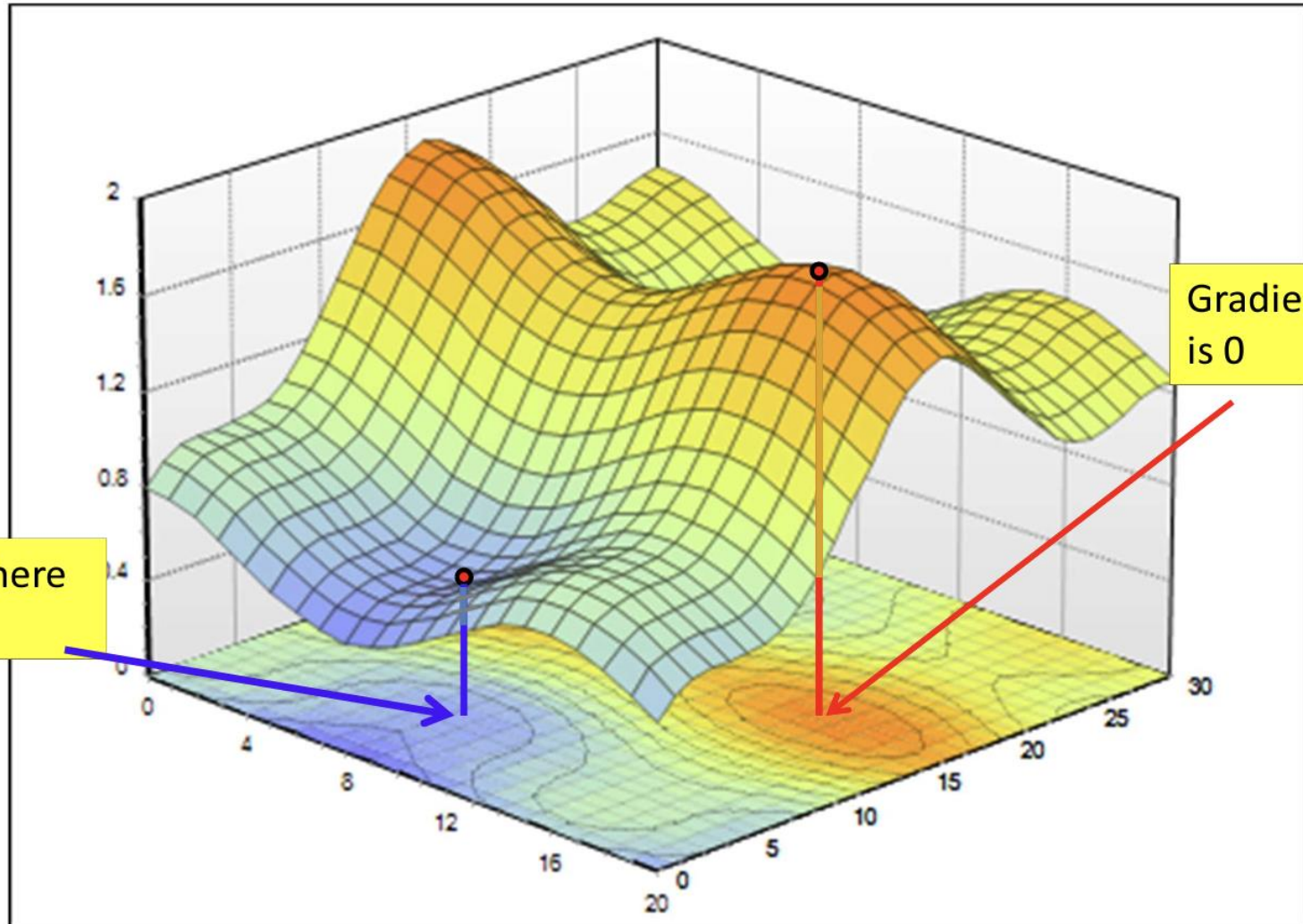
Gradient



Gradient



Gradient



The Hessian

- The Hessian of a function $f(x_1, x_2, \dots, x_n)$ is given by the second derivative

$$\nabla_x^2 f(x_1, \dots, x_n) := \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdot & \cdot & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdot & \cdot & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdot & \cdot & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Select all that are true about derivatives of a scalar function $f(X)$ of multivariate inputs

✓ 0

At any location X , there may be many directions in which we can step, such that $f(X)$ increases

0

The direction of the gradient is the direction in which the function increases the fastest

0

$y=f(X)$ is a scalar function of an $N \times 1$ column vector variable X . What is the shape of the gradient of y with respect to X .



Scalar

0

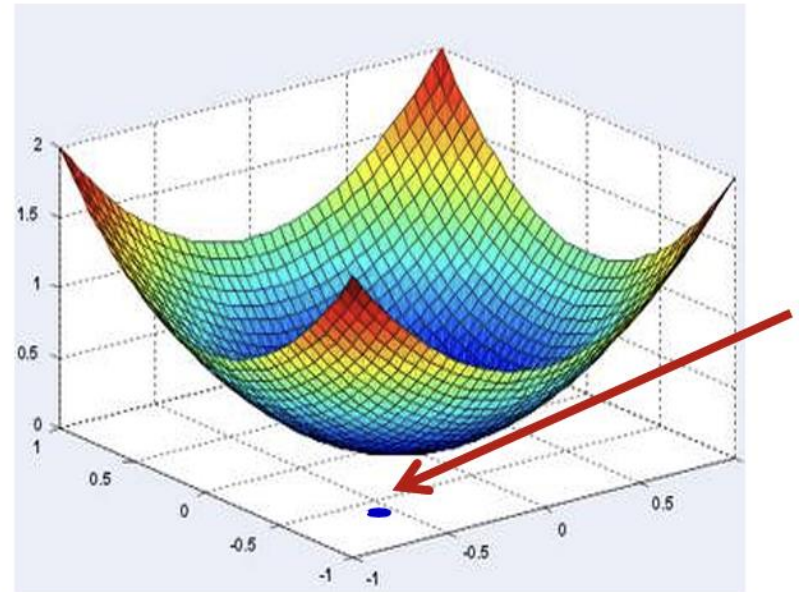
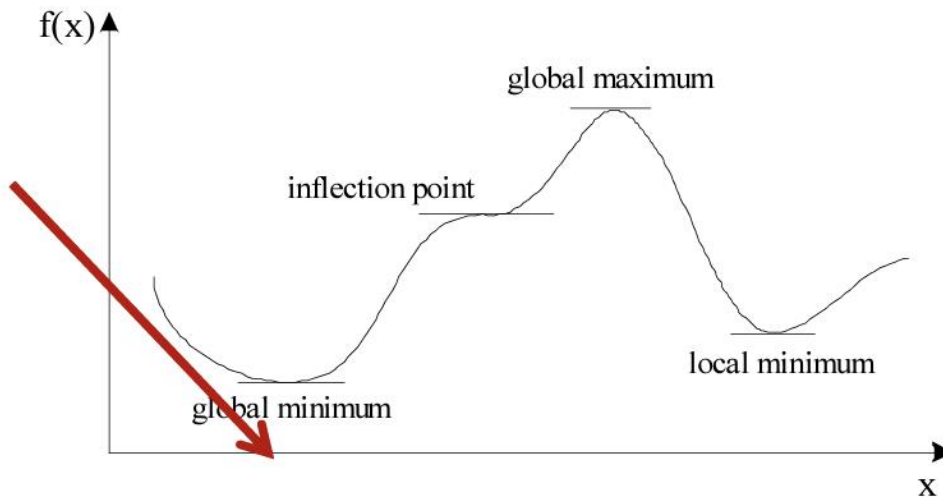
$N \times 1$ column vector

0

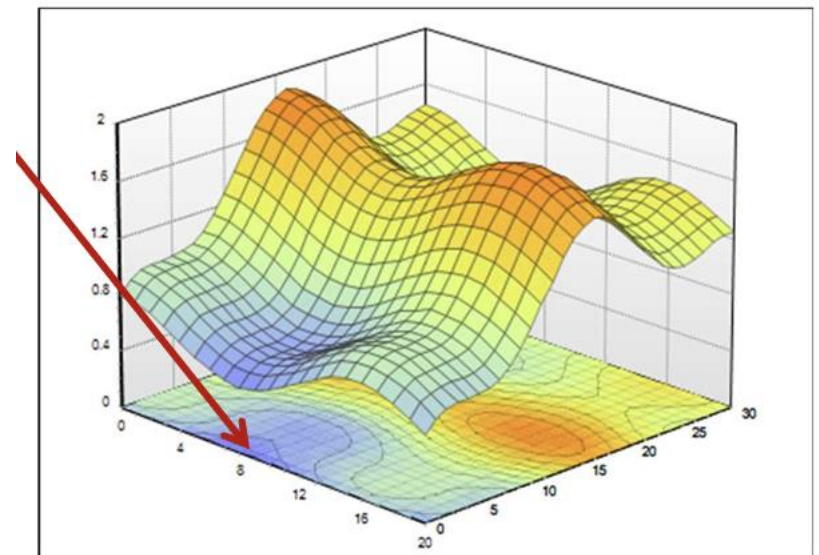
$1 \times N$ row vector

0

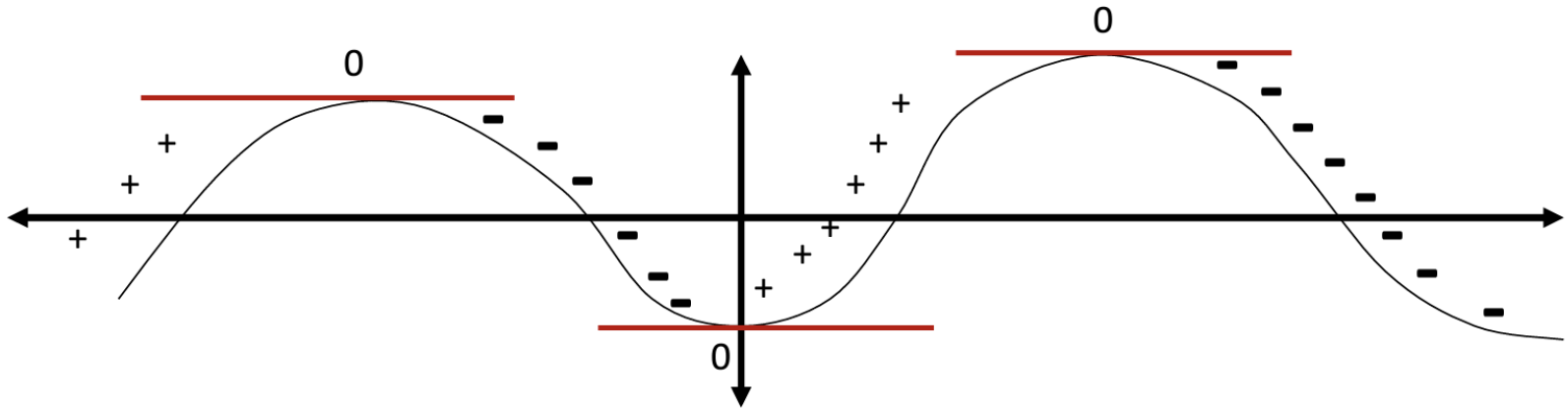
The Problem of Optimization



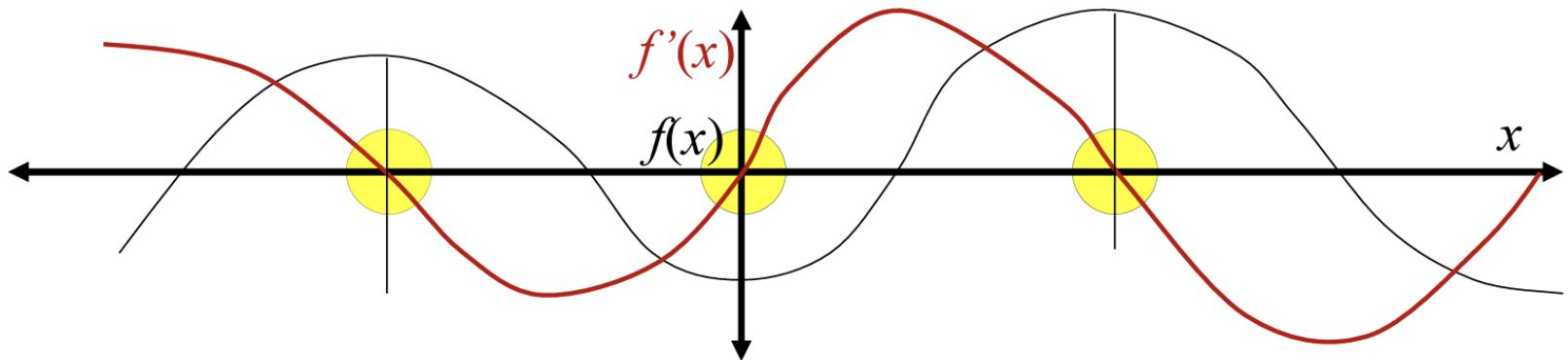
- General problem of optimization:
Given a function $f(X)$ of X
- Find the value of X where $f(X)$ is minimum



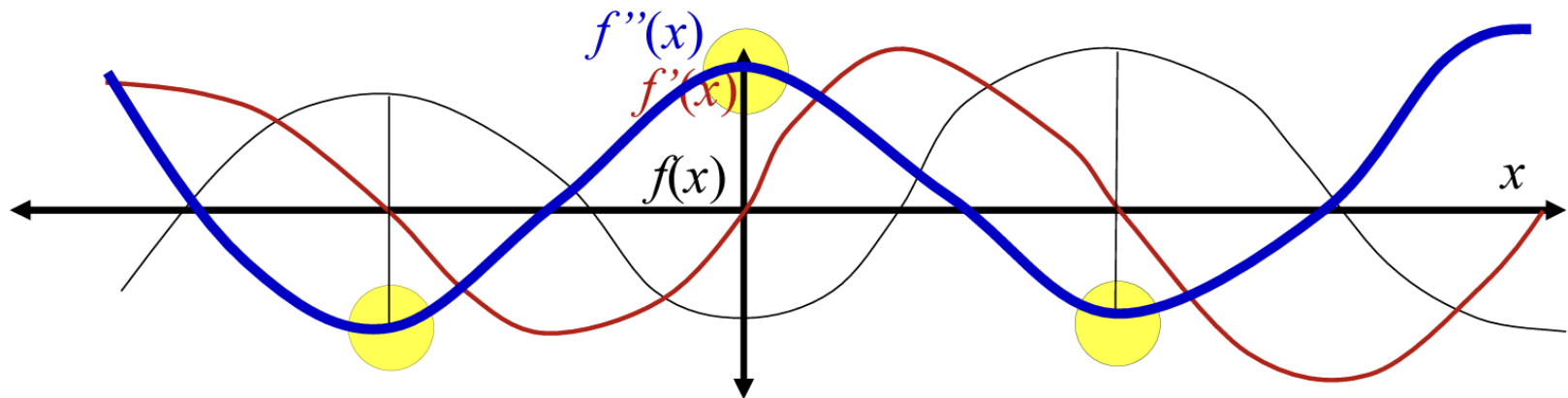
Finding the Minimum of a Function



- Find the value X at which $f'(X) = 0$
- Both maxima and minima have zero derivative

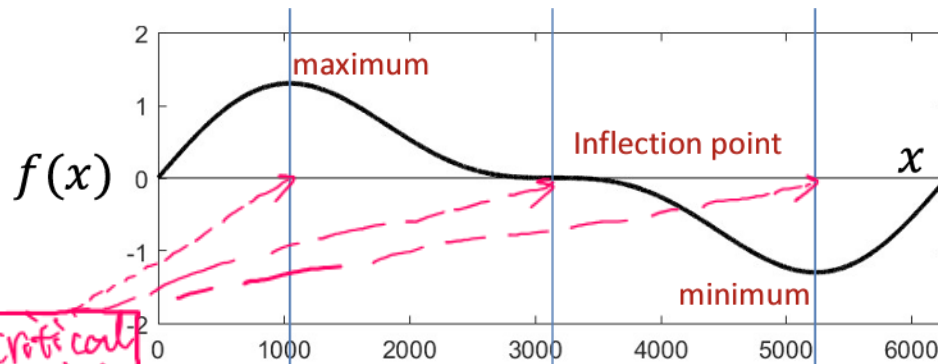


Derivative of the Derivative of the Curve



- Find the value X at which $f'(X) = 0$
- Both maxima and minima have zero derivative
- The second derivative $f''(X)$ is **negative** at **maxima** and **positive** at **minima**!

A Note on Derivatives of Functions of Single Variable



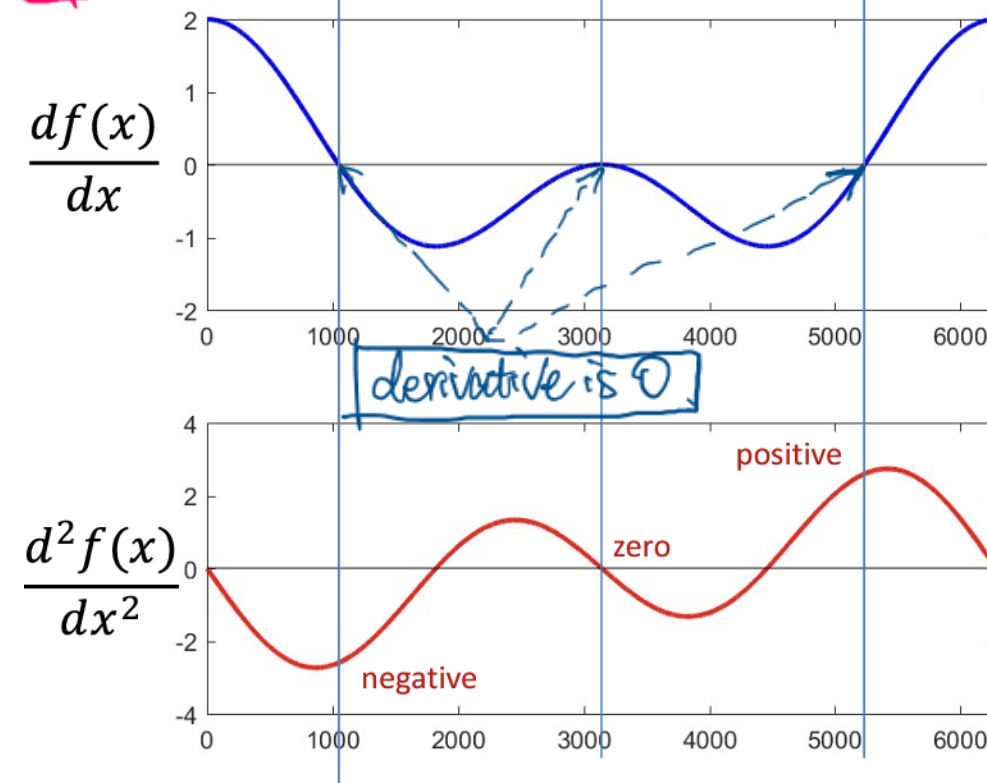
All locations with 0 derivative are critical points

- Local maxima, local minima, or inflection points

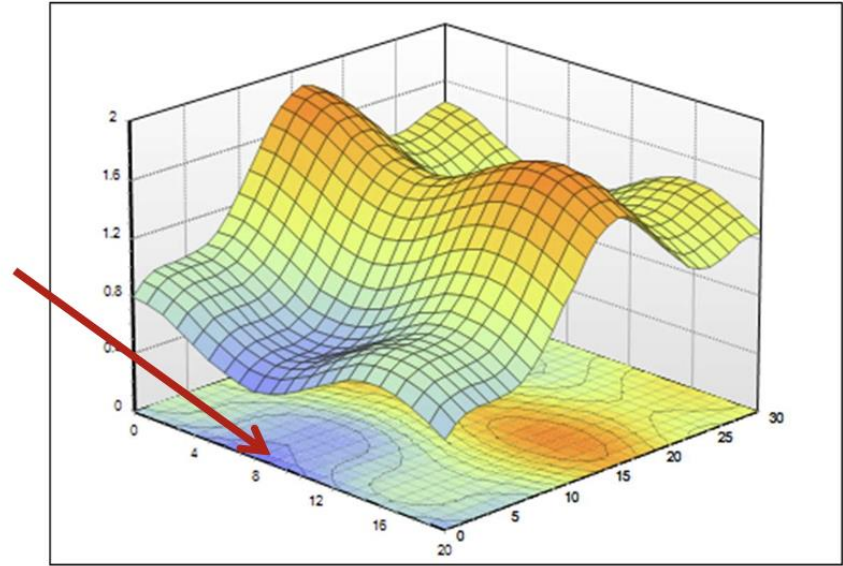
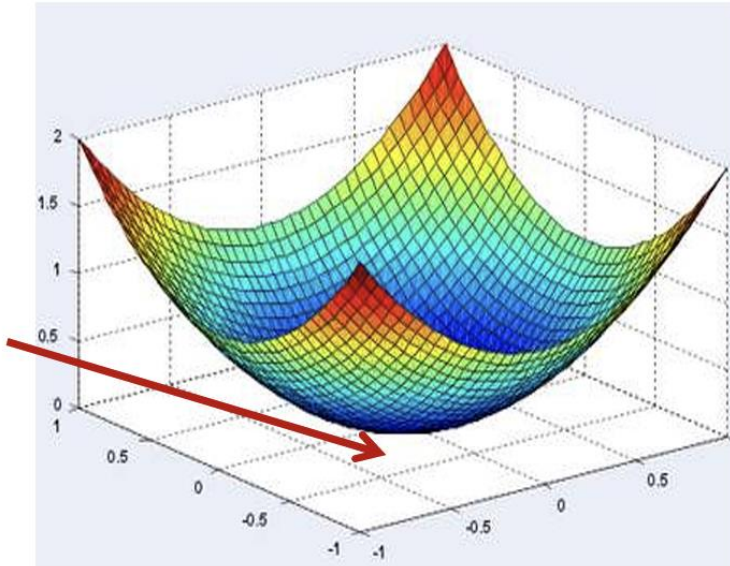
The second derivative

- > 0 at minima
- < 0 at maxima
- $= 0$ at inflection point

More complicated for functions of multiple variables...



What about Functions of Multiple Variables?



For smooth functions, at the maximum/minimum, the gradient is 0

- Really tiny shifts will not change the function value

Find the location where the gradient is 0

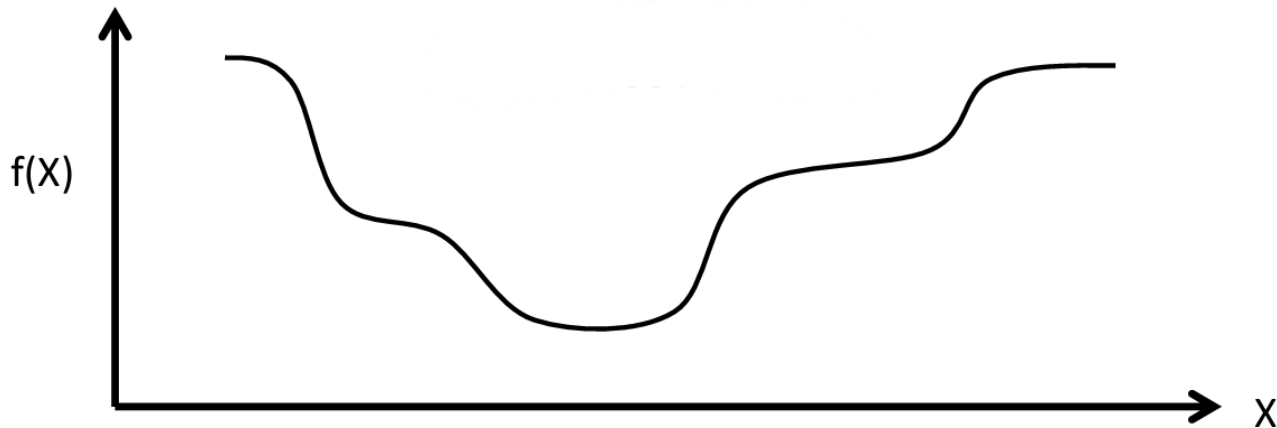
Unconstrained Minimization of Function (Multivariate)

- Solve for the value where the gradient equals to zero

$$\nabla_X f(X) = 0$$

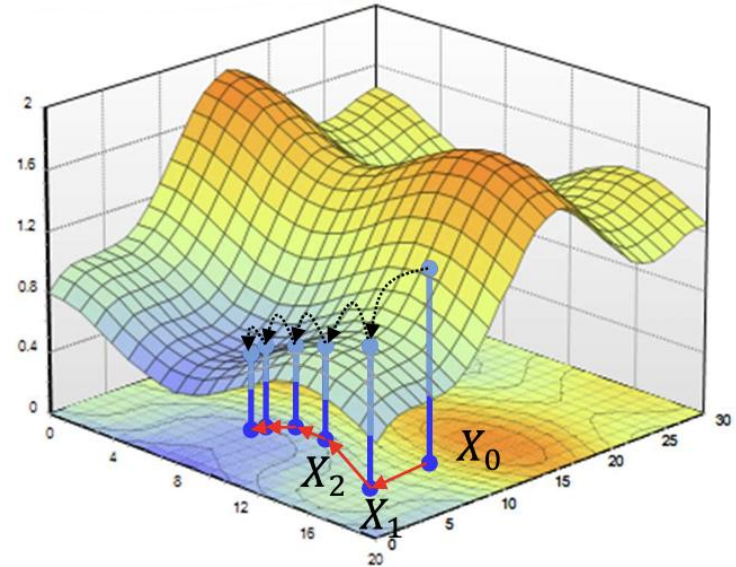
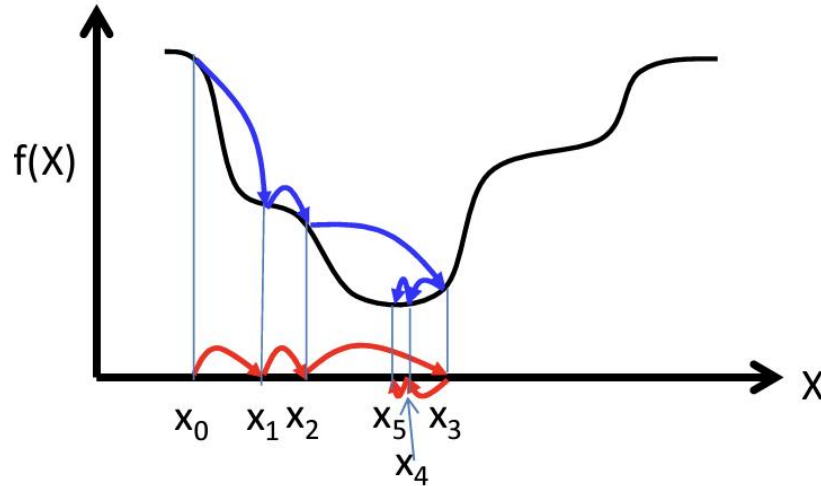
- Compute the Hessian Matrix $\nabla_X^2 f(X)$ at the candidate solution and verify that
 - Hessian is positive definite (eigenvalues positive) -> to identify local minima
 - Hessian is negative definite (eigenvalues negative) -> to identify local maxima

Closed Form solutions are not Always Available



- Often it is not possible to simply solve $\nabla_x f(X) = 0$
- The function to minimize/maximize may have an intractable form
- In these situations, iterative solutions are used
- Begin with a “guess” for the optimal X and refine it iteratively until the correct value is obtained

Iterative Solutions



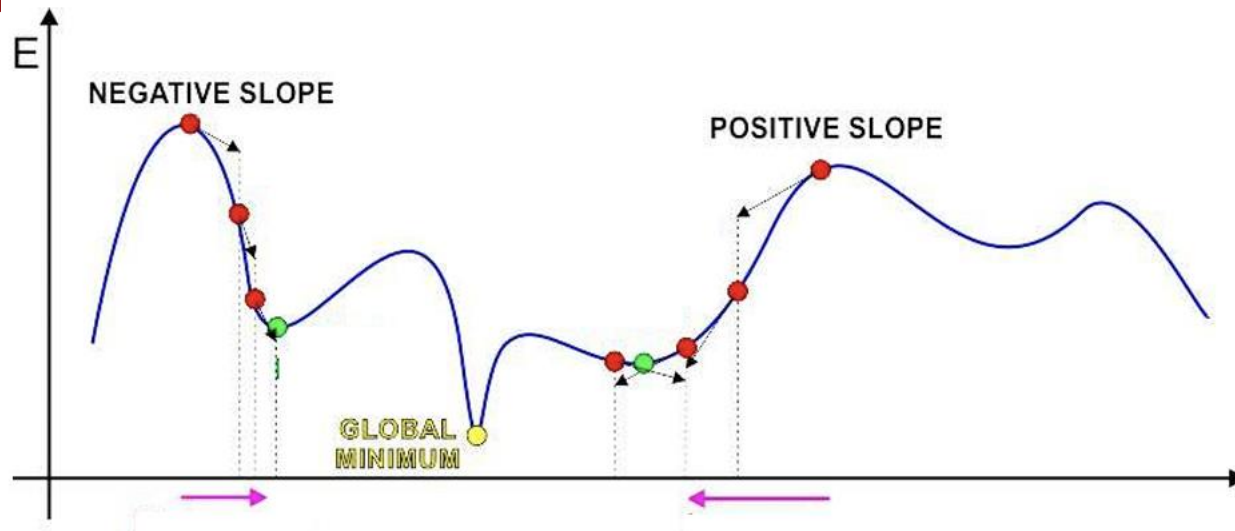
Iterative solutions

- Start from an initial guess X^0 for the optimal X
- Update the guess towards a (hopefully) “better” value of $f(X)$
- Stop when $f(X)$ no longer decreases

Problem:

- Which direction to step in
- How big must the steps be

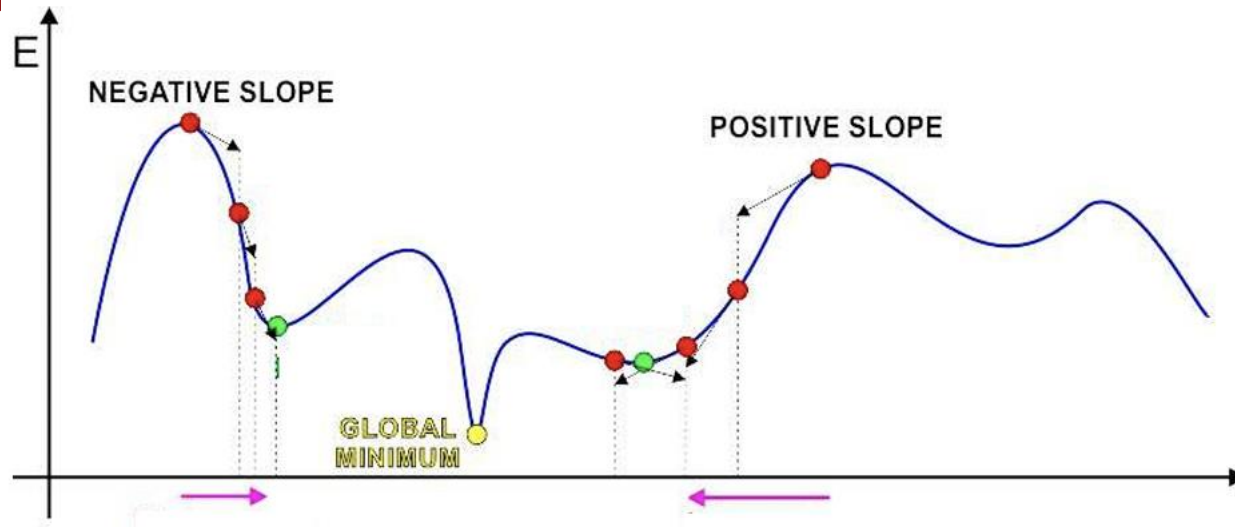
The Approach of Gradient Descent



Iterative solution:

- Start at some point
- Find direction in which to shift this point to decrease error
 - Using the gradient of the function
 - A negative derivative $\rightarrow$ moving right decreases error
 - A positive derivative $\rightarrow$ moving left decreases error
- Shift point in this direction

The Approach of Gradient Descent



Iterative solution: Trivial algorithm

- Initialize X^0
- While $f'(X^k) \neq 0$:

$$X^{k+1} = X^k - \eta^k f'(X^k)$$

- If $\text{sign}(f'(X^k))$ is positive:

$$X^{k+1} = X^k - \Delta X$$

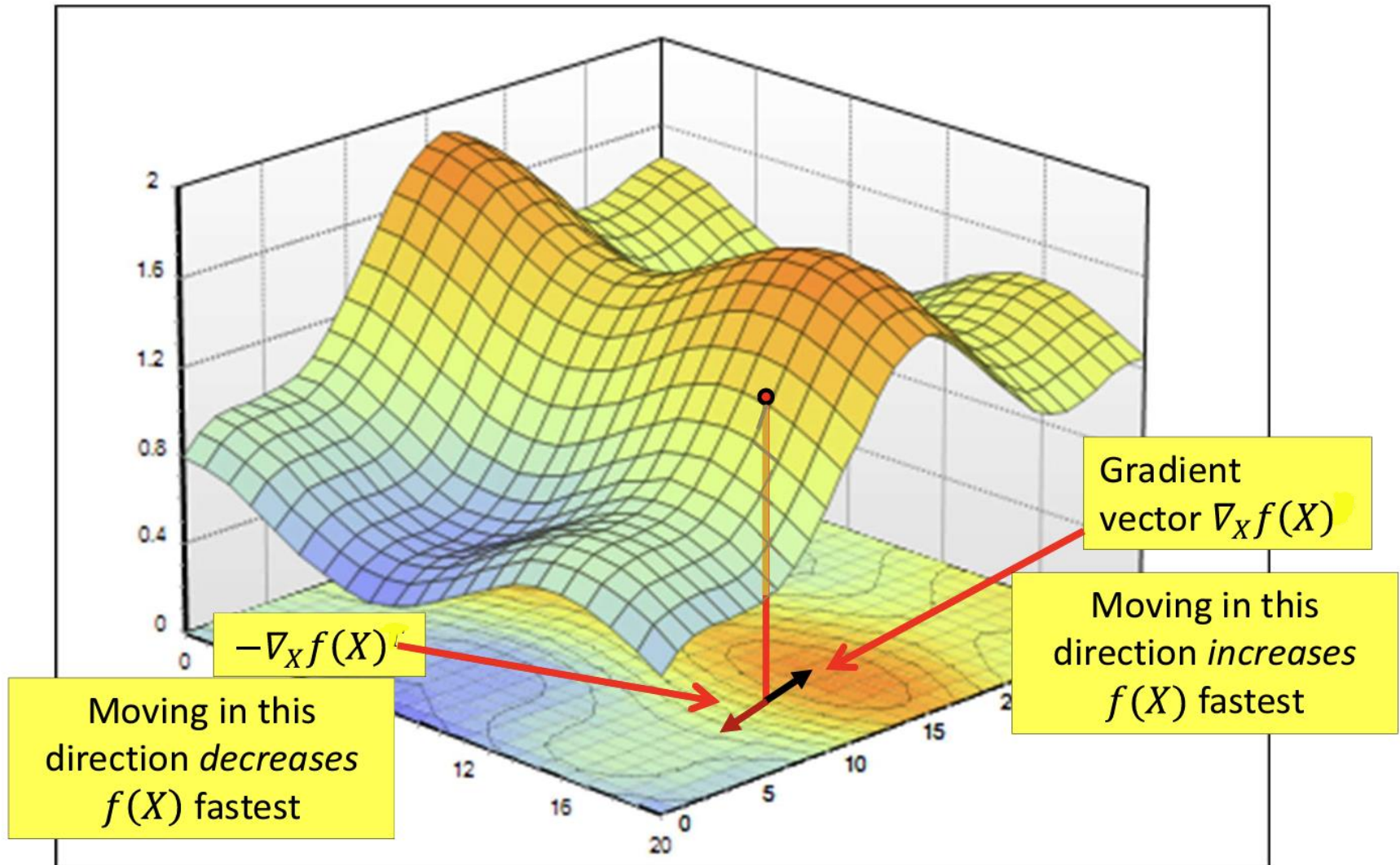
- Else:

$$X^{k+1} = X^k + \Delta X$$

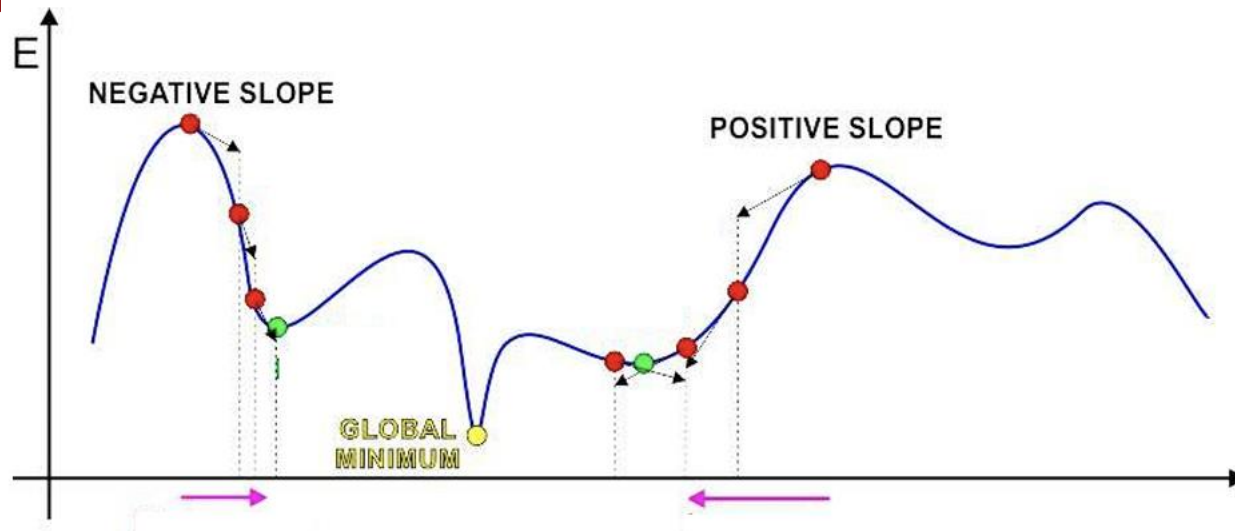
- What should ΔX be?

- $\Delta X = \eta^k f'(X^k)$, where η^k is the step size

Gradients of Multivariate Functions



The Approach of Gradient Descent



Iterative solution: Trivial algorithm

- Initialize X^0
- While $f'(X^k) \neq 0$:

$$X^{k+1} = X^k - \eta^k f'(X^k)$$

Gradient Descent/Ascent (Multivariate)

The gradient descent/ascent method to find the minimum or maximum of a function f iteratively

- To find a maximum, move in the direction of the gradient:

$$X^{k+1} = X^k + \eta^k \nabla_X f(X^k)$$

- To find a minimum, move exactly opposite the direction of the gradient:

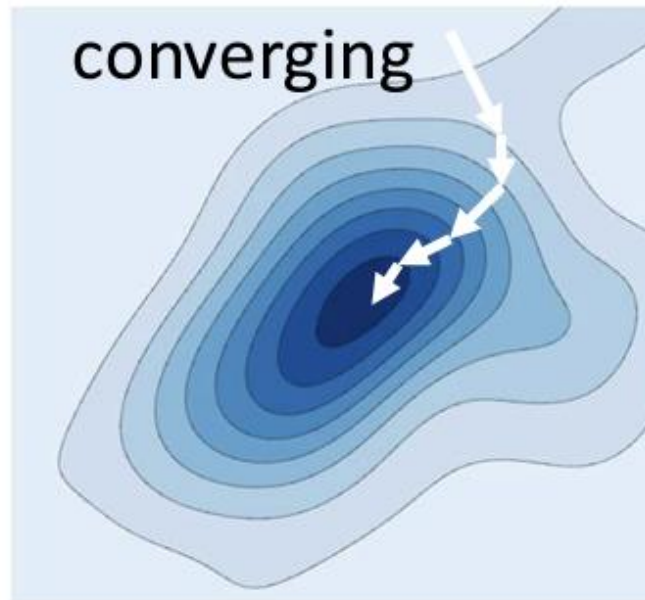
$$X^{k+1} = X^k - \eta^k \nabla_X f(X^k)$$

- Many solutions to choosing step size η^k
- Why do we want to yield the largest change in the objective function?

Convergence of Gradient Descent

An iterative algorithm is said to **converge** to a solution if the value updates arrive at a fixed point

- Where the gradient is 0 and further updates do not change the estimate



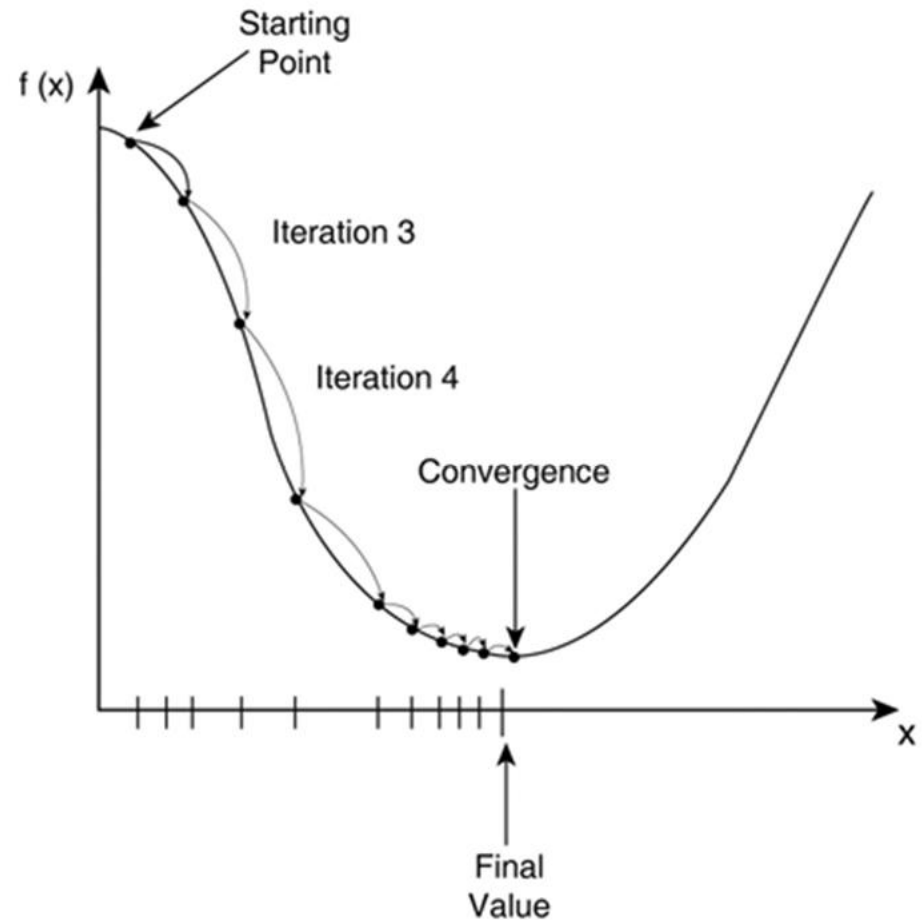
Gradient Descent Convergence Criteria

- The gradient descent algorithm converges when one of the following criteria is satisfied

$$|f(X^{k+1}) - f(X^k)| < \epsilon_1$$

- Or

$$\|\nabla_X f(X^k)\| < \epsilon_2$$



Overall Gradient Descent Algorithm

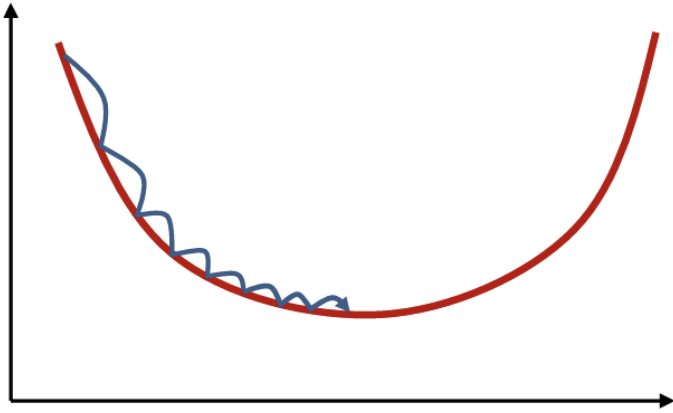
Initialize:

- X^0
- $k = 0$

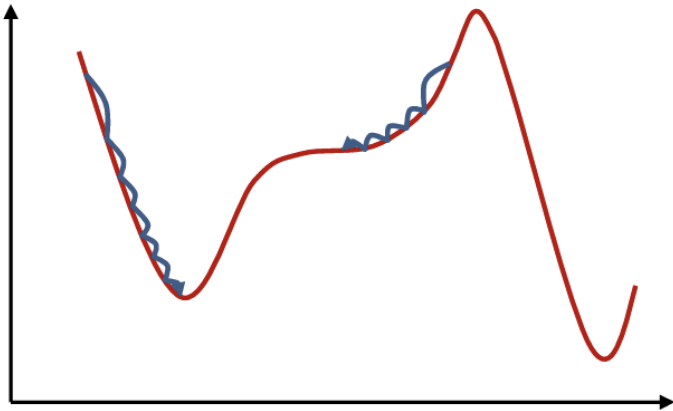
While $|f(X^k) - f(X^{k-1})| > \epsilon$ do:

- $X^{k+1} = X^k - \eta^k \nabla_X f(X^k)$
- $k = k + 1$

Convergence of Gradient Descent



- For **appropriate step size**, for convex (bowl - shaped) functions gradient descent will always find the minimum.
- For non-convex functions it will find a local minimum or an inflection point



$y = f(X)$ is a scalar function of an $N \times 1$ column vector variable X . Starting from $X = X^0$, in which direction must we move in the space of X , to achieve the maximum decrease in $f()$?



Exactly in the direction of the gradient of $f(X)$ at X^0

0

Exactly perpendicular to the direction of the gradient of $f(X)$ at X^0

0

Exactly opposite to the direction of the gradient of $f(X)$ at X^0

0

Supervised Learning Problem Restated

- Given a training set of input-label pairs

$$\mathcal{D} = \{(X_i, Y_i) | i = 1, 2, \dots, n\}$$

- Select a model class parametrized by θ
- Quantify the error on the i^{th} sample $error(f(X_i; \theta), Y_i)$
- Calculate the empirical risk

$$Loss(\theta) = \frac{1}{N} \sum_{i=1}^n error(f(X_i; \theta), Y_i)$$

- Estimate the parameters to minimize the empirical risk

$$\hat{\theta} = \arg \min_{\theta} Loss(\theta)$$

Gradient Descent to Learn a Supervised Learning Model

Initialize:

- θ^0
- $k = 0$

While $|Loss(\theta^k) - Loss(\theta^{k-1})| > \epsilon$ do:

- $\theta^{k+1} = \theta^k - \eta^k \nabla_{\theta} Loss(\theta^k)$
- $k = k + 1$

Gradient Descent for Linear Regression

```
from torch.nn.functional import mse_loss

# we need to convert the training and test
X_train = torch.tensor(X_train.to_numpy())
y_train = torch.tensor(y_train.to_numpy())

theta, theta_prev = random_initialization()
while abs(J(theta) - J(theta_prev)) > conv_threshold:
    loss = mse_loss(prediction, y_train)
    loss.backward()
    theta_prev = theta
    theta = theta_prev - step_size * theta.gradient
    theta = theta.require_grad_()
```

“HW1 autograd equivalent”: cannot use nn.sequential():

You should reuse most of the code in class

```
threshold = 1e-5
step_size = 0.1
# initialize theta
theta = torch.tensor(np.array([2.,1.]), requires_grad = True)
theta_prev = torch.tensor(np.ones(2,), requires_grad = True)
iter = 0

while torch.linalg.norm(theta - theta_prev) > threshold:
    theta_prev = theta
    # we first calculate the prediction on the training dataset
    prediction = f(X_train, theta)
    loss = mse_loss(prediction, y_train)
    loss.backward()

    with torch.no_grad(): # No need to track history for weight updates
        theta = theta_prev - step_size * theta.grad
        # attaching the gradient after the update step
        theta = theta.requires_grad_()

    iter += 1
    if iter % 100 == 0:
        print('Iteration %d. MSE: %.6f' % (iter, loss))
```

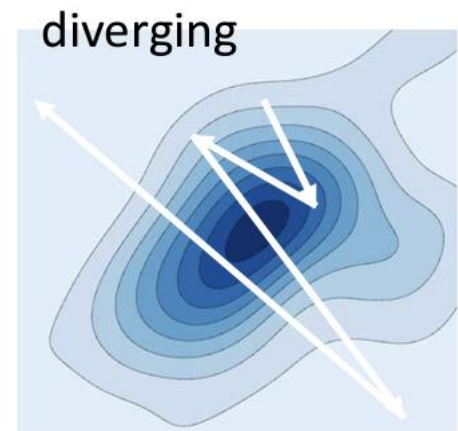
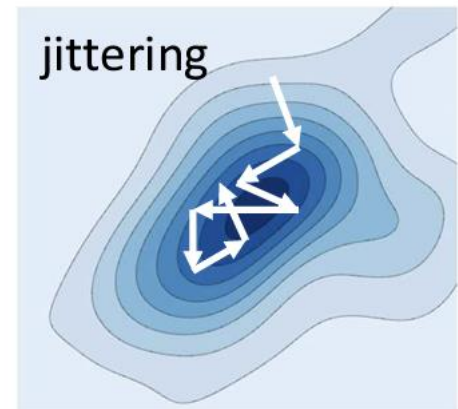
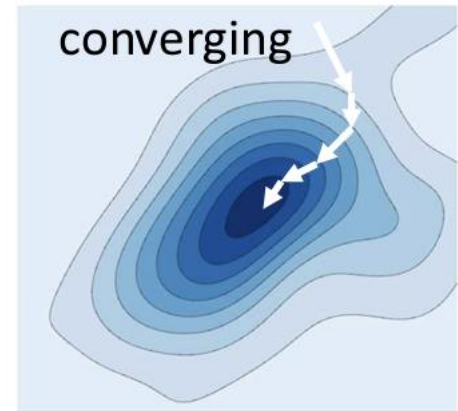
Convergence of Gradient Descent

An iterative algorithm is said to **converge** to a solution if the value updates arrive at a fixed point

- Where the gradient is 0 and further updates do not change the estimate

The algorithm may not actually converge

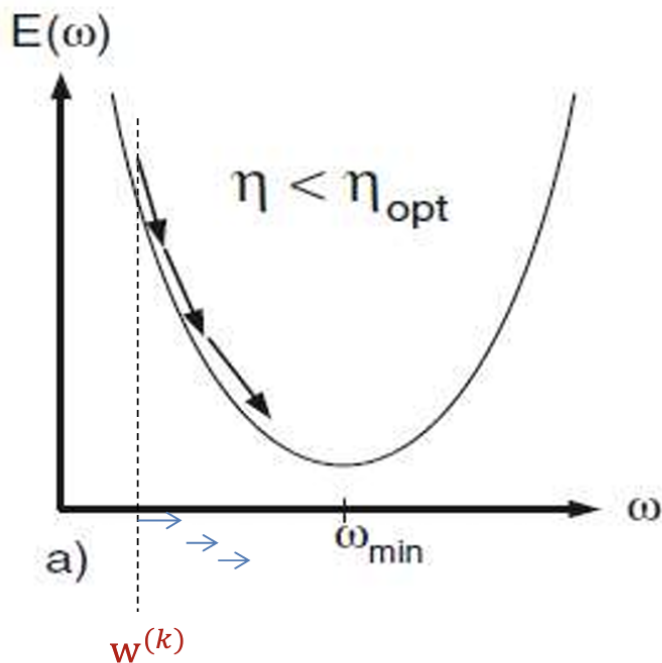
- It may jitter around the local minimum
- It may even diverge
- Conditions for convergence?



Convergence for Quadratic Surfaces

$$\text{Minimize } E = \frac{1}{2}aw^2 + bw + c$$

$$w^{(k+1)} = w^{(k)} - \eta \frac{dE(w^{(k)})}{dw}$$



Gradient descent with fixed step size η to estimate *scalar* parameter w

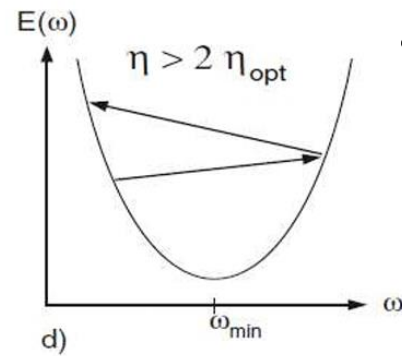
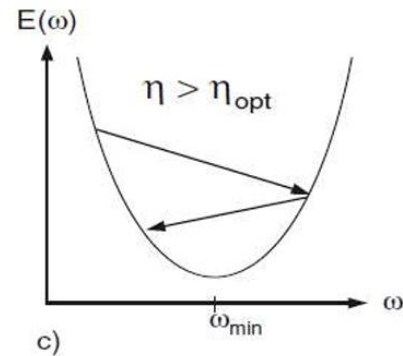
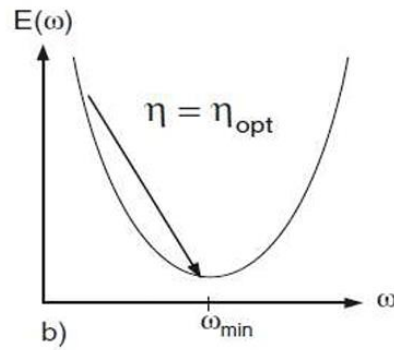
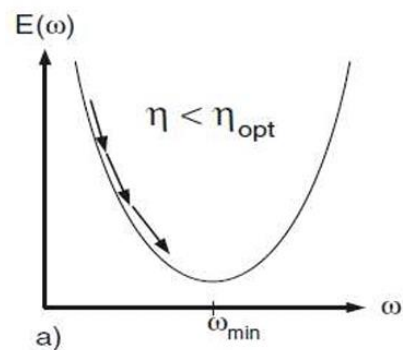
- Gradient descent to find the optimum of a quadratic starting from $w^{(k)}$
- Assuming fixed step size η
- What is the optimal step size η to get there fastest?
- Can arrive at the optimum in a single step using the optimal step size

$$\eta_{\text{opt}} = E''(w^{(k)})^{-1} = a^{-1}$$

With Non-Optimal Step Size

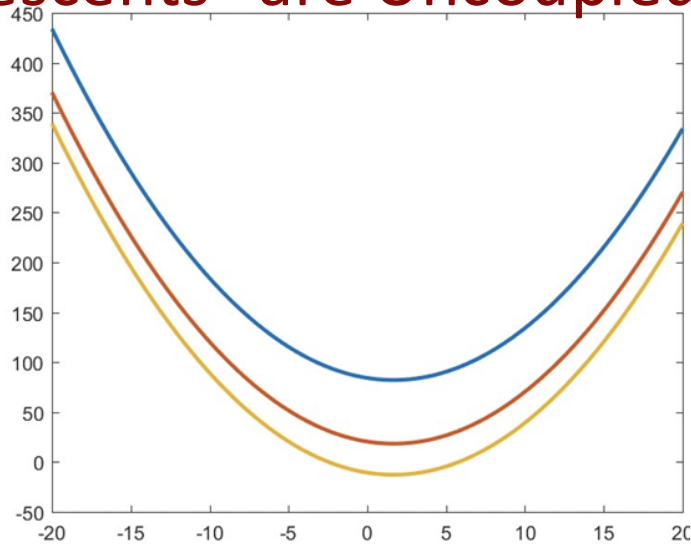
$$w^{(k+1)} = w^{(k)} - \eta \frac{dE(w^{(k)})}{dw}$$

Gradient descent with fixed step size η to estimate *scalar* parameter w

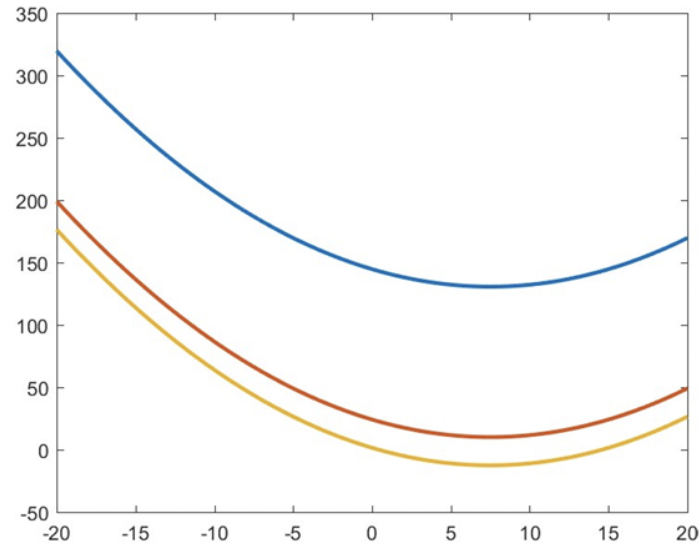


- For $\eta < \eta_{\text{opt}}$ the algorithm will converge monotonically
- For $\eta_{\text{opt}} < \eta < 2\eta_{\text{opt}}$ we have oscillating convergence
- For $2\eta_{\text{opt}} < \eta$ we get divergence

“Descents” are Uncoupled



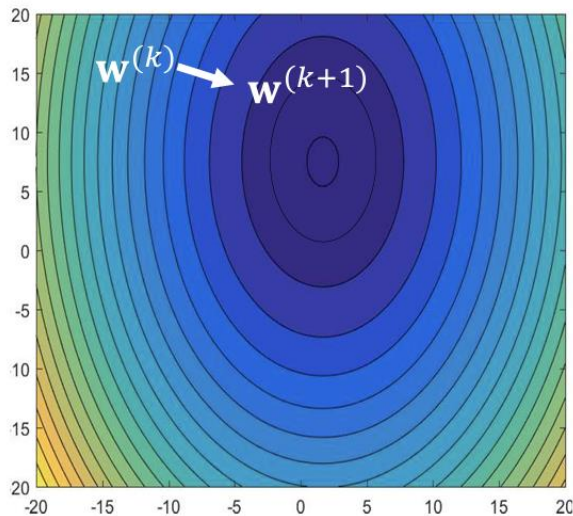
$$E = \frac{1}{2}a_{11}w_1^2 + b_1w_1 + c + C(\neg w_1)$$
$$\eta_{1,opt} = a_{11}^{-1}$$



$$E = \frac{1}{2}a_{22}w_2^2 + b_2w_2 + c + C(\neg w_2)$$
$$\eta_{2,opt} = a_{22}^{-1}$$

- The optimum of each coordinate is not affected by the other coordinates
- I.e. we could optimize each coordinate independently
- **Optimal learning rate is different for the different coordinates**

Vector Update Rule



$$\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \nabla_{\mathbf{w}} E^{\top}$$

$$w_i^{(k+1)} = w_i^{(k)} - \eta \frac{\partial E(w_i^{(k)})}{\partial w}$$

- Conventional vector update rules for gradient descent: update the entire vector against direction of gradient
- Note: Gradient is perpendicular to equal value contour
- The same learning rate is applied to all components

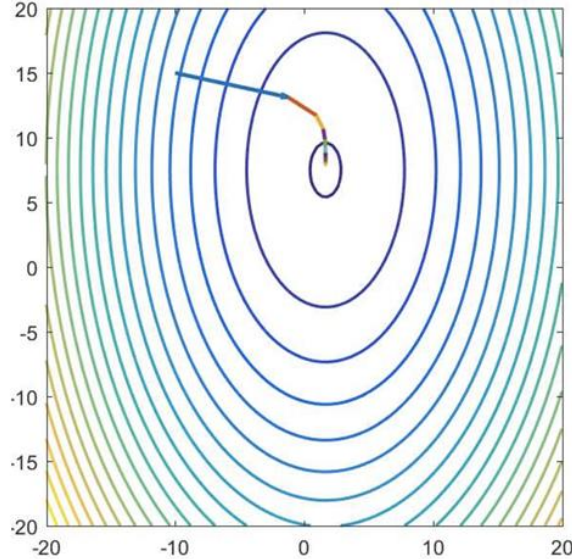
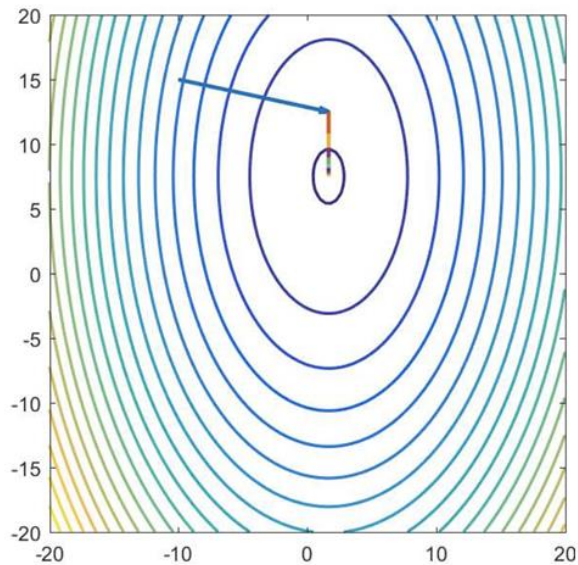
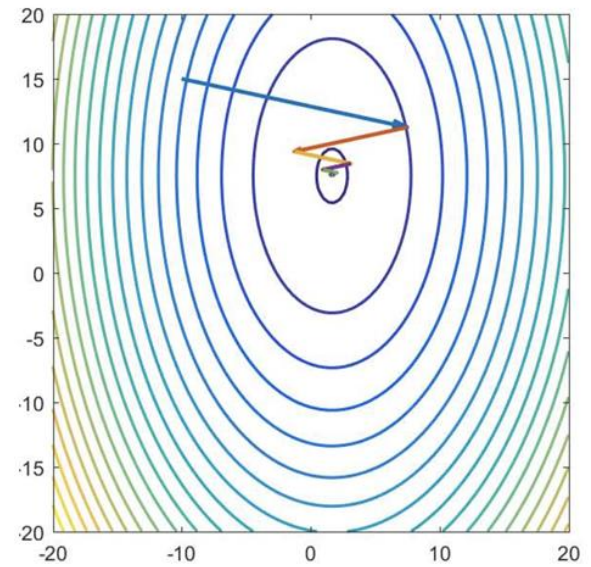
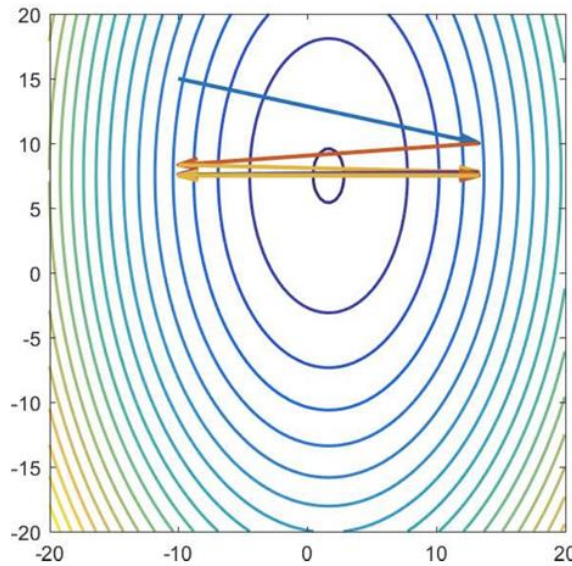
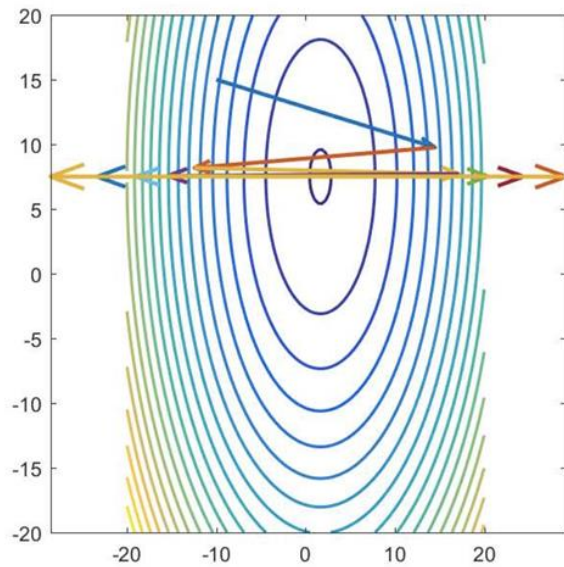
Problem with Vector Update Rule

$$\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \nabla_{\mathbf{w}} E^T$$

$$w_i^{(k+1)} = w_i^{(k)} - \eta \frac{\partial E(w_i^{(k)})}{\partial w}$$

$$\eta_{i,opt} = \left(\frac{\partial^2 E(w_i^{(k)})}{\partial w_i^2} \right)^{-1} = a_{ii}^{-1}$$

Dependence on Learning Rate



- $\eta_{1,opt} = 1; \eta_{2,opt} = 0.33$
- $\eta = 2.1\eta_{2,opt}$
- $\eta = 2\eta_{2,opt}$
- $\eta = 1.5\eta_{2,opt}$
- $\eta = \eta_{2,opt}$
- $\eta = 0.75\eta_{2,opt}$